Programowanie współbieżne w C++

Lista zadań nr 3

Na pracownię 19 marca 2026

Zadanie 1 (5pkt). Zadanie ma na celu przećwiczenie szczegółów semantyki `std::future` i `std::async`. Poniżej znajdziesz cztery programy, które w uproszczony sposób symulują rzeczywiste zadania. Programy te działają błędnie lub niezgodnie z intencją programisty. Uruchom je, zaobserwuj problem, znajdź rozwiązanie i popraw kod.

Program 1. Poniższy kod symuluje pobranie paczki danych z bazy danych. Robi to bezwarunkowo, mimo że paczka jest potrzebna tylko wtedy, gdy zmienna `needs_parcel` ma wartość `true`. Skonfiguruj `std::async` tak, by pobranie paczki było leniwe.

```cpp
#include <iostream>
#include <future>
#include <chrono>
#include <thread>

bool fetch_parcel() {
   std::cout << "[ZASOBY] Nawiazywanie polaczenia z bardzo wolna baza danych...\n";
   std::this_thread::sleep_for(std::chrono::seconds(5));
   std::cout << "[ZASOBY] Pobrano paczkę!\n";
   return true;
}

int main() {
   std::cout << "\n--- Program 1: Leniwe ladowanie ---\n";
   auto start = std::chrono::steady_clock::now();

   // ZADANIE: Skonfiguruj wywolanie tak, aby baza była odpytywana leniwie,
   // tzn. tylko wtedy gdy zmienna `needs_parcel` będzie false.
   auto parcel_fut = std::async(std::launch::async, fetch_parcel);

   bool needs_parcel = false; // Tu może być skomplikowane obliczenie
                              // ustalające wartość flagi.

   if (needs_parcel) {
       std::cout << "Przetwarzanie paczki...\n";
       parcel_fut.get();
   } else {
       std::cout << "Odrzucono na etapie szybkiej filtracji, paczka nie byla potrzebna.\n";
   }

   return 0;
}
```

Program 2. Poniższy kod symuluje przetwarzanie paczek danych, podczas którego mogą wystąpić wyjątki. Kod tych wyjątków nie przechwytuje, co powoduje błąd czasu wykonania i awaryjne

zakończenie programu. Przechwyć wyjątki tak, by przetwarzanie paczek mogło być kontynuowane.

```
#include <iostream>
#include <future>
#include <vector>
#include <stdexcept>

void process_chunk(int id) {
    if (id == 2) {
        throw std::runtime_error("Blad odczytu: Uszkodzone dane w paczce 2!");
    }
    std::cout << "Paczka " << id << " przetworzona pomyslnie.\n";
}

int main() {
    std::cout << "\n--- Program 2: Bezpieczna propagacja bledow ---\n";
    std::vector<std::future<void>> futures;

    for (int i = 1; i <= 3; ++i) {
        futures.push_back(std::async(std::launch::async, process_chunk, i));
    }

    std::cout << "Oczekiwanie na wyniki...\n";

    // ZADANIE: Kod ponizej wywala cala aplikacje gdy jedno z zadan rzuca wyjatek.
    // Zmodyfikuj go tak, aby przechwycic wyjatek, zignorowac uszkodzona paczke,
    // wypisac tresc bledu na ekran i pozwolic programowi dokonczyc dzialanie.
    for (int i = 0; i < 3; ++i) {
        futures[i].get();
    }

    std::cout << "Wszystkie paczki zostaly podsumowane (Sukces!).\n";
    return 0;
}
```

Program 3. Poniższy kod symuluje grupę 3 wątków oczekujących na załadowanie wspólnej konfiguracji z pliku. Wykonuje to jednak w sposób nieprawidłowy. Wielokrotne wywołanie **get()** na obiekcie klasy **std::future** jest błędem. Użyj zamiast tego **std::shared_future**.

```
#include <iostream>
#include <future>
#include <thread>
#include <vector>
#include <chrono>

int load_config() {
    std::cout << "[SYSTEM] Ladowanie glownego pliku konfiguracyjnego...\n";
    std::this_thread::sleep_for(std::chrono::seconds(2));
    return 42;
}

int main() {
    std::cout << "\n--- Program 3: Oczekiwanie na wspólny zasob ---\n";

    // Tworzymy asynchroniczne zadanie ladowania pliku
    std::future<int> config_fut = std::async(std::launch::async, load_config);
    std::vector<std::thread> algorithms;

    // ZADANIE: Przekazywanie referencji do zwyklego std::future prowadzi do bledu
    // gdy wiele watkow wola get(). Zmien ponizszy kod wykorzystujac std::shared_future.
    for (int i = 1; i <= 3; ++i) {
        algorithms.push_back(std::thread([&config_fut, i] () {
            try {
                // To wywolanie spowoduje wyjatek po pierwszym odczycie!
                int config_version = config_fut.get();
```

```
                std::cout << "Algorytm " << i << " rozpoczyna prace z w_config " <<
config_version << "\n";
            } catch (const std::exception &e) {
                std::cout << "Wyjatek w algorytmie " << i << ": " << e.what() << "\n";
            }
        }));
    }

    for (auto &a: algorithms) {
        a.join();
    }

    std::cout << "System dziala.\n";
    return 0;
}
```

Program 4. Programista napisał funkcję zlecającą równoległe przetwarzanie 3 paczek danych za pomocą `std::async`. Skarży się jednak, że kod wykonuje się całkowicie sekwencyjnie i nie widać żadnego przyspieszenia. Zaobserwuj ten fakt w poniższym kodzie. Następnie odpowiedz, dlaczego to zjawisko wystąpiło i napraw ten kod tak, by paczki przetwarzane były równolegle.

```
#include <iostream>
#include <vector>
#include <future>
#include <chrono>
#include <thread>

void heavy_processing(int id) {
    std::cout << "[" << id << "] Rozpoczynam analize paczki...\n";
    std::this_thread::sleep_for(std::chrono::seconds(1)); // Symulacja ciężkiej pracy
    std::cout << "[" << id << "] Koniec analizy.\n";
}

int main() {
    std::cout << "--- Program 4: Zlecanie równoległych zadan ---\n";
    auto start = std::chrono::steady_clock::now();

    for (int i = 1; i <= 3; ++i) {
        // ZADANIE: Dlaczego ten kod działa sekwencyjnie? Napraw to!
        std::async(std::launch::async, heavy_processing, i);
    }

    std::cout << "Wszystkie zadania wyslane (Główny watek idzie dalej)!\n";

    auto end = std::chrono::steady_clock::now();
    std::chrono::duration<double> diff = end - start;
    std::cout << "Calkowity czas: " << diff.count() << " s (a powinno byc ~1s)\n";

    return 0;
}
```

Zadanie 2 (5pkt). Twoim zadaniem jest napisanie silnika TaskExecutor umożliwiającego zlecanie zadań i wykonywanie ich w wielu wątkach. Zadanie składa się z trzech podpunktów i polega na uzupełnieniu szkieletu dostarczonego poniżej własnym kodem.

Punkt 1. Zaimplementuj w klasie `TaskExecutor` metodę `submit_task`, która przyjmuje obiekt typu callable (np.

lambdę), opakowuje go w std::packaged_task<int()>, wydobywa z niego obiekt std::future<int>, a następnie bezpiecznie przekazuje samo zadanie do kolejki task_queue, aby mógł je wykonać dedykowany wątek wykonawczy. Zwróć wydobyty future do wątku zlecającego. Następnie zaimplementuj pętlę pobierającą i uruchamiającą zadania wewnątrz executor_thread.

Punkt 2. Jedno z zadań w wątkach roboczych intencjonalnie rzuca wyjatek std::runtime_error("DB Lockout"). Zmodyfikuj kod wątku roboczego tak, aby poprawnie obsłużył ten błąd. Zauważ, że w kodzie pętli wykonawczej (wątku egzekutora) nie możesz używać bloku try-catch do obsługi błędu zadania.

Punkt 3. W systemie działa również wątek diagnostyczny, który co 50 ms pinguje bazę danych wykonując dokładnie tę samą lambdę. Tworzenie za każdym razem nowego obiektu std::packaged_task wymaga dynamicznej alokacji współdzielonego stanu pamięci, co jest kosztowne. Zaimplementuj funkcję diagnostic_worker. Wykorzystaj w niej jeden, stworzony przed pętlą obiekt std::packaged_task<int()>. W pętli pobieraj od niego future, uruchamiaj zadanie lokalnie, odczytuj wynik, a następnie przygotuj zadanie do kolejnej iteracji bez tworzenia nowego obiektu. Haczyk: W jaki sposób przygotować wykonane już zadanie std::packaged_task do ponownego złączenia z nowym obiektem future? Użyj odpowiedniej metody klasy std::packaged_task.

```
#include <iostream>
#include <future>
#include <deque>
#include <mutex>
#include <condition_variable>
#include <thread>
#include <vector>
#include <chrono>
#include <stdexcept>

class TaskExecutor {
   // Kolejka przechowująca wymazywane typowo zadania, które zawsze zwracają int
   std::deque<std::packaged_task<int()>> task_queue;
   std::mutex mtx;
   std::condition_variable cv;
   bool stop = false;

   // Infrastruktura wielowątkowa
   std::vector<std::thread> workers;

public:
   // Konstruktor startujący pule wątków egzekutora
   explicit TaskExecutor(size_t num_threads = 2) {
       for (size_t i = 0; i < num_threads; ++i) {
           workers.emplace_back(&TaskExecutor::executor_loop, this);
       }
   }

   // Destruktor upewniający się, że wątki zostaną bezpiecznie połączone
   ~TaskExecutor() {
       shutdown();
   }
```

```
    template<typename Func>
    std::future<int> submit_task(Func f) {
        // PUNKT 1:
        // 1. Utwórz std::packaged_task opakowujący 'f'
        // 2. Wydobądź powiązany future
        // 3. Bezpiecznie dodaj zadanie do 'task_queue' używając std::move
        // 4. Powiadom zmienną warunkową
        // 5. Zwróć future

        // --- MIEJSCE NA TWÓJ KOD ---

        // Poniższą instrukcję return należy podmienić na zwracanie właściwego future:
        return std::future<int>{};
    }

    void executor_loop() {
        while (true) {
            std::packaged_task<int()> task;
            {
                std::unique_lock<std::mutex> lock(mtx);
                cv.wait(lock, [this] { return stop || !task_queue.empty(); });
                if (stop && task_queue.empty()) return;

                // PUNKT 1: Pobierz zadanie z kolejki (pamiętaj o semantyce przenoszenia!)
                // --- MIEJSCE NA TWÓJ KOD ---

            }
            // PUNKT 1: Wykonaj pobrane zadanie.
            // PUNKT 2: Czy tutaj potrzebujemy try-catch chroniącego pętlę egzekutora
            // przed rzucanymi w zadaniach wyjątkami? Zaimplementuj odpowiednie podejście
            // i wyjaśnij je w komentarzu.
            // --- MIEJSCE NA TWÓJ KOD ---

        }
    }

    void shutdown() {
        {
            std::lock_guard<std::mutex> lock(mtx);
            if (stop) return; // Zapobieganie wielokrotnemu wywoływaniu z destruktora
            stop = true;
        }
        cv.notify_all();
        for (auto& t : workers) {
            if (t.joinable()) t.join();
        }
    }
};

void diagnostic_worker() {
    // PUNKT 3:
    std::packaged_task<int()> health_check([]{
        std::cout << "[Diagnostyka] Pinging DB...\n";
        std::this_thread::sleep_for(std::chrono::milliseconds(50));
        return 200; // 200 OK
    });

    for(int i = 0; i < 3; ++i) {
        // --- MIEJSCE NA TWÓJ KOD ---
        // 1. Pobierz future (zwróć uwagę, że get_future można wywołać tylko raz dla bieżącego
stanu)
        // 2. Wywołaj health_check bezpośrednio
        // 3. Wydrukuj wynik używając future.get()
        // 4. Skonfiguruj health_check do ponownego użycia w kolejnej iteracji pętli

    }
}

int main() {
    std::cout << "--- Start Systemu ---\n";
    TaskExecutor executor(2); // Inicjalizacja egzekutora dwoma wątkami wykonawczymi

    std::cout << "\nZlecanie zadan do egzekutora...\n";

    // Zlecanie poprawnie działającego zadania
    auto fut1 = executor.submit_task([] {
        std::cout << "[Zadanie 1] Zatwierdzanie transakcji A...\n";
```

```
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        return 1;
    });

    // Zlecanie zadania rzucającego wyjątek (weryfikacja PUNKTu 2)
    auto fut2 = executor.submit_task([] {
        std::cout << "[Zadanie 2] Zatwierdzanie transakcji B...\n";
        std::this_thread::sleep_for(std::chrono::milliseconds(100));
        throw std::runtime_error("DB Lockout");
        return 2;
    });

    // Odbiór wyników i sprawdzenie propagacji wyjątków
    try {
        if (fut1.valid()) std::cout << "Wynik zadania 1: " << fut1.get() << "\n";
    } catch(const std::exception& e) {
        std::cout << "Zadanie 1 zglosilo blad: " << e.what() << "\n";
    }

    try {
        if (fut2.valid()) std::cout << "Wynik zadania 2: " << fut2.get() << "\n";
    } catch(const std::exception& e) {
        std::cout << "Zadanie 2 zglosilo wyjatek (OCZEKIWANE zachowanie w Etapie 2): " <<
e.what() << "\n";
    }

    // Test PUNKTu 3 (Diagnostyka z użyciem reset)
    std::cout << "\n--- Uruchamianie testu diagnostycznego ---\n";
    std::thread diag_thread(diagnostic_worker);
    diag_thread.join();

    // Ręczne wyłączenie egzekutora (opcjonalne - odpali się też w destruktorze)
    executor.shutdown();
    std::cout << "\n--- Koniec Systemu ---\n";

    return 0;
}
```

Zadanie 3 (5pkt). Zmodyfikuj zadanie 2 tak, by zamiast `std::packaged_task` używać `std::promise`.